Appendix A MACHXL

A.1 MACHXL user's handbook

MACHXL is an integrated environment for creating designs for AMD MACH 3 and 4 series programmable logic components. These are high density electrically erasable CMOS programmable logic devices, based on the macrocell concept. MACHXL cannot be used to develop designs for the MACH 1 and 2 series, nor can it be used to develop designs for simple PLDs such as the 22V10. However, the programming language used by MACHXL is a superset of PALASM which is a standard language used by AMD for programming all their PLD products. By obtaining the appropriate PALASM compiler and avoiding MACH specific constructs, the entire range of products can be programmed. The MACHXL software development product is low cost and enables all the design principles and applications that are used as examples in this book to be implemented.

MACHXL enables the user to edit, compile, simulate and program a range of cPLDs from the same user interface. The use of the interface is relatively self explanatory and the user should be able to deduce the actions required. Detailed information can be obtained from the full user manual. Although PALASM and its superset MACHXL are high level languages, they should not be confused with the majority of high level languages designed to execute on a personal computer. HLLs for programmable logic are hardware description languages (HDLs) and effectively each statement executes in parallel with all other statements. Execution does not begin at the top of the program listing, then sequentially executing each statement in turn until the end of the listing is reached. This must be remembered when writing HDL programs and any timing requirements of the design have to be explicitly implemented. The MACHXL package consists of five different programs which are executed sequentially once the source text file has been created. These are:

1. **Parser** Checks the syntax of the statements in the source text file (file extension .PDS) and if error free creates an intermediate file with a .TRE extension. A listing file is always created.

2. **Boolean post processor** Takes the intermediate file and converts complex statements such as CASE and IF THEN ELSE into equivalent Boolean equations.

3. **STATE syntax expander** If the state machine constructs have been used (not recommended for new MACHXL designs) this program converts them into Boolean equations. The Boolean post processor will then be executed a second time.

4. **Logic minimiser** Uses the file outputs from the Boolean post processor and reduces them to their minimum form, to maximise the efficiency of their implementation on to the architecture of the selected devices.

5. **Fitter** Fitting is a complex process which takes the output from the minimiser, allocates logic equations to logic hardware, determines the logic programming and routing required and creates the JEDEC file used for programming.

Designs can fail at any of the five stages, although the most common problems are the use of incorrect syntax in the source file which will be identified by the parser, and insufficient hardware resources being available during the fitting stage.

A.1.1 PALASM syntax

Where the name PALASM is used, the explanation applies equally to PALASM and MACHXL programs. If MACHXL is used, it only applies to MACHXL programs and should not be used in PALASM programs. Unlike most high level languages, PALASM does not use statement terminators, so the user must be alert to the differences. PALASM uses the beginning of the next valid statement as the terminator for the previous one and this can cause some confusion until the user becomes familiar with this concept.

A.1.2 Comments

The beginning of a comment is indicated by a semicolon and is terminated by the return key. Therefore, comments do not automatically extend over several lines and operate in a similar way to comments in assembly language programs.

Example

```
pin   11   value1  ;This is the comment for the statement at the
                   ;beginning of the line and continues over
                   ;three lines.
```

A.1.3 Program file structure

The PALASM program file structure is standardised for all designs and consists of three sections:

1. Declaration section containing nine fields which describe the design.

2. Design sections. There are two possible types of design sections:

   a) state machines; b) equations.

   At least one section must be present and may contain multiple instances of these constructs.

3. Simulation section.

A.1.4 The declaration section

The declaration section consists of a number of different fields the first six of which are identified by reserved words as illustrated below:

title	The title of the design goes here
pattern	The revision number of the PCB goes here.
revision	Version number of the software.
author	Designer's name goes here.
company	Company name goes here.
date	Date of version goes here.

These six fields effectively act as comments and should be used to keep track of revision numbers and PCB to be used for production amd maintenance purposes. They have no effect on the PLD design. The seventh field is optional and is used to describe any special functions the PLD may have, such as a security fuse, or a turbo bit. It is identified by the reserved word OPTIONS. The eighth field is required and is hardware specific, describing the target PLD and the allocation of signal names to pin numbers. It is identified by the reserved word CHIP, followed by a design name (user selectable) and then the target cPLD. Available target identifiers are:

MACH435
MACH445
MACH465
MACH335

Following this, each of the pins to be used is identified on a separate line using the reserved word PIN, followed by the signal name and then the pin type. If physical connection to a pin is not required and an internal macro cell can be used, the reserved word NODE can be used instead. Available pin types are listed below and can be used in combination. The sequence used in combinations is not important.

```
3volt          ; 3 volt drive level
5volt          ; 5 volt drive level
cmos_level     ; CMOS voltage levels
combinational  ; combinational output
registered     ; registered output
latched        ; latched output
pinfbk         ; pin feedback
cmbfbk         ; combinational feedback
regfbk         ; registered feedback
group          ; sequence of signals grouped together
high           ; active high outputs
low            ; active low outputs
open_drain     ; open drain output for different voltage or current levels
delayclk       ; delayed clock signal
ttl_level      ; TTL voltage levels.
```

An example of the use of the CHIP reserved word is given below.

Example

```
CHIP   decoder_design   MACH445 ;This selects device
                                ;Now the pin definitions follow
pin   2  value1
pin   3  value2
pin   4  input_6
```

The ninth and final field is also optional and is a string substitution function. If a sequence of characters is to be used repeatedly within a program, or some simplification

of statements is required to aid understanding, then the string reserved word can be used. This is followed by the name of the string and the text to be substituted in single quotation marks. If any constants are required within the logic description, for example, to describe the output LED patterns for a seven segment display, it is good programming practice to replace them with a constant name during the logic description. This is achieved through the use of a string substitution section which associates constant names with constant values. During the compilation process, constant names are replaced with the corresponding constant values. The single quotation marks are not included. The use of a string substitution section improves the readability of the programming listing and is considered to be good programming practice.

```
string state1          '#b0001'
string state2          '#b0010'
string last_state      '#b00'
string enable_1        'value1 * input_6'
```

Once the program header has been completed, the logic specification or description can be entered. A complete example of a declaration section is given below:

```
title      dram controller
pattern    pcb_revision_A_1996
revision   first
author     R C Seals
Company    University of Greenwich
Date       13th July 1996

options    turbo = on
           security = off

chip       d_controller    mach445

;Pin definition section

pin  6     clk2
pin  7     pclk
pin  8     ads
pin  9     clk2mio
pin  10    pa2
pin  12    iready
pin  13    ras0p
pin  14    sel1
pin  15    ras1p
pin  16    refin

;strings substituted

string     qads    '(ads * pclk * /iready)'
string     ras0n   '((ras0p + ras1) * /sel1)'
```

A.1.5 Key punctuation

Below is a list of the key punctuation symbols with some simple examples of how they can be used.

```
()    ;groups logic      A = B*(C + D)
;     ;Anything after the semicolon is treated as a comment until the
      ;next return.
..    ;Double full stop indicates a range of values.
?     ;Floating PIN or NODE assignment to avoid specifying the pin
      ;number before fitting.
```

A.1.6 Number bases

Three number bases are used, which are identified by the preceding single character:

```
binary      #b101     (binary representation of 5)
decimal     #d5
hexadecimal #h5
```

The hash symbol, #, is used to distinguish the numbers from user specified signal names.

A.1.7 User specified signal names

As well as the predefined keywords and key punctuation symbols, the user is able to specify signal names. Signal names are associated with either inputs or outputs and are used to hold logic values which may vary during the operation of the logic function implemented. The variable names can contain up to 14 alphanumeric characters plus the underscore character. Upper and lower case letters are treated identically. The space character cannot be used in signal names, nor can any other punctuation character.

A.1.8 Examples of valid variable names

```
state
upper_level
data
val_1
val_2
val270
```

A.1.9 Examples of invalid variable names

```
numb 11
numb-11
data>count
```

A.1.10 Examples of 'bad' design names

Although the following are syntactically correct signal names, they are regarded as bad programming style, because they do not convey any information about the significance of the signal within the design.

```
Burt
Jons_work
rubbish1
```

A.1.11 Examples of 'good' design names

```
4_bit_adder
address_decoder
state_machine
```

A.1.12 Pin vector definitions

It is not necessary to give separate variable names to each pin. Groups of pins with a common functions, such as a data bus, can be defined as a one dimensional vector.

Example

```
PIN  13:16   /state[3:0] registered
```

This allocates a group of 4 outputs to 4 successive pins, and is equivalent to:

```
PIN  13 /state[3] registered
PIN  14 /state[2] registered
PIN  15 /state[1] registered
PIN  16 /state[0] registered
```

Not all the pins on the PLD chosen have to be defined. Individual vector states can be accessed as indicated above, or as groups.

Example

```
drive = motor * fuel * state[3]
state[3:2] = state[1:0]
```

Vector notation is supported for combinational and registered designs and is particularly useful for signals grouped and used as a bus system. Vectors can be defined in two ways:

1) during the pin definition,
2) using the reserved word VECTOR to group existing signals.

An example of defining a vector during the pin definitions is given below.

```
pin data[7..0]          ; this is an 8-bit vector
```

Below is an example of defining a vector using the VECTOR reserved word

```
vector addr := [A3, A2, A1, A0]  ; this is a 4-bit vector
```

Using the vector notation is a shorthand way of manipulating multiple signals.

```
addr[3..0] := data[3..0]
```

which is the same as:

```
addr[3] := data[3]
addr[2] := data[2]
addr[1] := data[1]
addr[0] := data[0]
```

Similarly:

```
addr[3..0] := data[7..4] * opcode[1..4]
```

is the same as:

```
addr[3] := data[7] * opcode[1]
addr[2] := data[6] * opcode[2]
addr[1] := data[5] * opcode[3]
addr[0] := data[4] * opcode[4]
```

A.2 Logic specification

The logic specification is a description of the logic function to be implemented by the PLD and consists of assignments, logic equations and high level language constructs such as IF THEN ELSE.

A.2.1 Basic assignments

Assignments are identified by the equals character and can be made to variables or vectors, but they must have previously been defined. The normal meaning of the equals symbol is used, with the variable on the left hand side of the equals sign becoming equivalent to the right hand side. If the equals sign on its own is used, the assignment is to be implemented using combinational logic, that is, without registers and clock signals. If the equals sign is preceded by a colon, sequential logic is used, that is registered logic. The right hand side can be a signal variable, a constant name or a number. For combinational variables the changes happen instantaneously. If a registered output is being used, then the output is updated at the next rising edge clock pulse.

The following assignment operators can be used:

=	assignment for combinational outputs
:=	assignment for registered outputs
*=	assignment for latched outputs

Note that registered outputs are clocked on the rising edge of the clock pulse when they take on the new value. Latched outputs become newly valid on the rising edge of the clock pulse, remain changeable throughout the clock pulse, and are latched to a stable value on the falling edge of the clock pulse.

Example

```
C = A * (D + E)*/D
state[0] = A
state[1] = C*state[3]
```

Multiple vectors can also be simultaneously assigned, as illustrated in the example below:

Example

```
state[3:0] = vector[0.3]* vect[3.0]
```

which is equivalent to:

```
state[3] = vector[0] * vect[3]
state[2] = vector[1] * vect[2]
state[1] = vector[2] * vect[2]
state[0] = vector[3] * vect[0]
```

Vectors can also be assigned a number up to the maximum value representable by the number of bits, N, in the vector, such that the maximum number is 2^N-1. Variables can also be assigned numbers, but it can only be 0 or 1.

Example

```
C = 0    ; If a number base is not defined
B = 1    ; It is treated as decimal

vector[3:0] = 6;
```

This is equivalent to:

```
vector[3:0] = #b0110
```

or

```
vector[3] = #b0
vector[2] = #b1
vector[1] = #b1
vector[0] = #b0
```

A.2.2 Logic equations

Almost any Boolean statement can be implemented, the main limitations are those of the number of inputs in the AND gates in the AND-OR array which limits the number of variables that can be ANDed. For the 22V10 this is 22, with each variable being available as its true or complement form. The number of inputs to the OR gate in the AND-OR array limits the number of product terms in any one equation. The implementation of other logic functions such as exclusive OR is achieved through basic AND and OR. The logic operations available in MACHXL are:

```
*    AND
+    OR
:+:  Exclusive OR
/    NOT
```

Normal precedence of logic operations is applied, but to ensure that the correct logic is implemented, open and close brackets can be used to group operations. Brackets can be nested to any level.

Examples of logic equations

```
A  =   C * D
B  =   A*(Q1:+:Q2) + /F + /F*(D0:+:/state)
```

The following basic Boolean operators are implemented for both combinational and registered designs and, in addition to those listed previously. Any Boolean equations have to be placed into the equations part of the design file, which is identified by the EQUATIONS reserved word. The equations section is assumed to continue until another section's reserved word is identified. It is possible in assignments to group variables together using the open and close brackets.

```
Z   =   A*B              ;  AND operation
Z   =   /(A*B)           ;  NAND operation
Z   =   A+B              ;  OR operation
Z   =   /(A+B)           ;  NOR operation
Z   =   A:+:B            ;  Exclusive OR operation
Z   =   (A*B)+(/A*/B)    ;  Two product terms
```

The same process can be performed with other language constructs using the BEGIN and END reserved words. An example is:

```
if (A = C * D) then
  begin
    B = A*(Q1:+:Q2) + /F + /F*(D0:+:/state)  ;These 5 statements
    vector[3] = #b0                         ;have been grouped
    vector[2] = #b1                         ;together.
    vector[1] = #b1
    vector[0] = #b0
  end
else
  begin
    B = #b0
  end
```

BEGIN and END act as group delimiters; they can be nested to any level and can be used in any program statement apart from assignment to effectively turn several statements into a pseudo single statement. It is important to remember that BEGINs and ENDs are always in pairs and, although they can be nested, there must always be the same number of BEGINs as there are ENDs.

Table A.1 Signal extensions

Signal extension	Explanation
.ACLK	Asynchronous clock to register
.CLKF	Synchronous clock (preferred)
.ALE	Asynchronous Latch Enable
.LE	Synchronous Latch Enable
.TRST	Output buffer enable
.RSTF	Reset
.SETF	Preset
.D	Data input to D type flip-flop
.T	Data input to Toggle flip-flop
.J	J input to JK flip-flop
.K	K input to JK flip-flop
.R	R input to SR flip-flop
.S	S input to SR flip-flop
.FB	Feedback from I/O pin

A.2.3 Signal extensions

Each output can have a variety of parameters controlled by using a signal extension. A full stop is used to separate the signal name from the signal extension.

```
signal_name.signal_extension
```

All the valid signal extensions are given in table A.1.

Examples

```
count.d          :=  value     ; D type flip-flop with
count.clkf        =  clk       ; synchronous clock
counter.t        :=  value     ; T type flip-flop with
counter.clkf      =  clk       ; synchronous clock
register_A.j     :=  1         ; JK flip-flop set up as
register_A.k     :=  1         ; a toggle
register_A.clkf   =  clk
```

A.3 High level language constructs

A number of other high level constructs are put together in the design file by the user, to describe the logic function to be implemented.

A.3.1 Structured statements

MACHXL supports two high level language constructs which can be used to combine the simpler logic equations and to provide some flexibility in the execution of the logic implemented. The two constructs are:

IF THEN with an optional ELSE

and

CASE

A.3.2 IF THEN ELSE

The IF THEN ELSE statement enables a conditional response to be made. That is, the response is dependent on the test condition, called the logic condition. The logic condition used to control the test must be contained within the open and close brackets and must be resolvable at all times. The statement can be used in two forms; the first is without the ELSE when the test is performed to decide whether to execute the logic equation or not:

```
IF (logic condition) THEN
  begin
    statement
  end
```

The second is with the ELSE in place, which is when one of two logic equations must be executed:

```
IF (logic condition) THEN
  begin
    statement 1
  end
ELSE
  begin
    statement2
  end
```

A complete example is given below.

Example

```
IF (drive) THEN
 begin
  drive = 0
```

```
  end
ELSE
  begin
    drive = 1
  end
```

Note that in this example the logic condition is simply a variable name. This is assumed to be:

```
IF (drive = 1) THEN
  begin
    drive = 0
  end
ELSE
  begin
    drive = 1
  end
```

Vectors can be used within the logic condition brackets as illustrated below.

Example

```
IF (vector[3..0]) THEN
  begin
    logic statement
  end
```

which is equivalent to ANDing all the vectors together as in:

```
IF (vector[3] * vector[2] * vector[1] * vector[0]) THEN
```

Specific vector values can also be checked for, as in the following:

```
IF (vector[3..0] = 6) THEN
  begin
    logic statement
  end
```

Conditions can be nested and the only restriction is that it must eventually evaluate to a boolean true or false:

```
IF ((vector[3..0] = 6) + (drive = 0)) THEN
  begin
    logic statement
  end
```

IF THEN ELSE statements can be nested, but care should be taken to ensure that the ELSE applies to the correct IF.

A.3.3 CASE construct

The CASE construct only operates on vectors, and matches the vector variable value against specific numerical values to select a statement to be executed. The reserved word CASE is used to begin the construct, followed by the vector variable contained within open and close brackets. This is followed by a BEGIN END pair which enclose the possible values that the vector variable can take. The possible values are called the guard conditions and identify which of the possible logic equations is actually executed. Each state possible is separated from the logic equation to be executed by a colon.

A.3.4 Example of syntax

```
CASE (vector[n..0])
    BEGIN
        value_0: begin statement_1 end
        value_1: begin statement_2 end
          .
          .
          .
        value_N: begin statement_N end
    END
```

The BEGIN and END surrounding each sequence of statements are compulsory. The number of values is determined by the size of the vector. A 2-bit vector has four values, a 3-bit vector has eight, and so on. A 1-bit vector is a special version of the case statement and is equivalent to an IF THEN ELSE statement.

Example

```
CASE (state[1..0])
    BEGIN
        0: begin vector[3] = 0 end
        1: begin vector[2] = 0 end
        2: begin vector[1] = 0 end
        3: begin drive = out_value end
    END
```

Not all values need to be specified, but if they are not, they must not occur, otherwise an undefined logic state may occur. If all the possible states of the vector are not defined, the compiler may not be able to compile the design, since internally it always expands the logic before looking for redundancies to condense it. Large vectors, 8-bit or more, create too many situations for the optimiser to cope with and it may fail. As a general rule, using large bit vectors within a CASE statement in PALASM, is not a good idea. Ranges of vectors can be denoted by using two full-stops as illustrated below.

```
CASE (state[3..0])
    BEGIN
        0..3: begin drive A + B end
        4..8: begin output = 0 end
        9:    begin vector[3..0] = 6 end
        10..15: begin A = A end
    END
```

Non-vector variables can be concatenated together by the comma, so they can be used in CASE statements as shown below.

```
CASE (A, B, C)
    BEGIN
        0: begin drive = 0 end
        1: begin A = B end
        5: begin vector[3:0] = state[0.3] end
    END
```

Multiple logic equations can be grouped together after each condition using a BEGIN END pair, as illustrated below.

```
CASE (state[1..0])
    BEGIN
        0: begin
             vector[3] = 0
             vector[6:4] = data[4:2]
           end
        1: begin vector[2] = 0 end
        2: begin vector[1] = 0 end
        3: begin
             drive = out_value
             data1 = in_value
           end
    END
```

It is common to use binary numbers in the conditions, as this corresponds more closely with the Boolean variable vectors being used. For the example below binary numbers have been used in the guard conditions.

```
CASE (state[1:0])
    BEGIN
        #b00: begin
                vector[3] = 0
                vector[6..4] = data[4..2]
              end
        #b01: begin vector[2] = 0 end
        #b10: begin vector[1] = 0 end
```

```
        #b11: begin
                drive = out_value
                data1 = in_value
              end
END
```

So far, only logic equations have been executed within the CASE statement, but in fact, any valid PALASM statement can be included, such as IF THEN ELSE, and further CASE statements. As with large bit vectors, nesting CASE statements can easily cause the optimiser to fail. If all the possible values of the variable being tested in the case statement are not explicitly defined, the logic generated may create unpredictable results should one of the undefined states occur. This can be avoided by the use of the reserved word OTHERWISE as illustrated below.

Example

```
CASE (state[3..0])
    BEGIN
        0: begin drive = A+B end
        4: begin output = 0 end
        15: begin A = A end
        OTHERWISE: begin C = A*B end
    END
```

A.4 STATE construct

The STATE construct used to implement state machines is not recommended for new designs using MACHXL. In PALASM, a state machine description begins with the reserved word

```
STATE
```

followed by the state of state machine

```
MOORE_MACHINE
MEALY_MACHINE
```

The difference between the two state machines is mainly in the outputs produced at each state. If the outputs at each state are always the same, that is, they never change even if the inputs are different, then it is a Moore state machine; whereas, if the outputs at each state can be different, depending on the values of the input signals, then it is a Mealy state machine. The same problems can be solved using Moore or Mealy state machines, the main difference being the number of states each machine requires. If for example a design has three states and, depending on the input values, each state can have three different sets of output values, then a Moore state machine would require nine states to be defined, whereas the Mealy state machine could be implemented with only three states. The three states of the Mealy machine could be implemented with only two

registers to give binary codings between 0 and 2, whereas the Moore machine would require three registers to give binary codings between 0 and 8. Therefore, if registers are limited, Mealy state machines should be used.

A.4.1 Moore state machine

A Moore state machine description consists of four sections:

1) the header definition
2) the state assignments
3) the state transitions
4) the input conditions controlling transitions

A.4.2 Header

The header description consists of two mandatory reserved words:

1) STATE
2) MOORE_MACHINE

followed by up to three default options:

1) OUTPUT_HOLD This indicates those output values which are to be maintained unchanged even when they are not explicitly controlled.

2) DEFAULT_OUTPUT This indicates the values of the specified signals if a state machine operation cannot be identified.

3) DEFAULT_BRANCH This indicates which state to move to (rather than specific output values), when the next state is not explicitly identifiable. There are three options possible with this:

    1) a user specified state,
    2) HOLD_STATE maintains the current state,
    3) NEXT_STATE moves to the next state in the user defined list of states in the states assignment section.

Example

```
state
moore_machine
default_branch S1
```

A.4.3 State assignments

This section, which comes immediately after the state header, describes the output values that each state is to have. There are three main techniques for allocating output values to states:

1) binary
2) Gray code
3) one hot encoding

A.4.4 Binary

Binary coding uses an incrementing binary sequence as illustrated next.

```
S0  = /Q2 * /Q1 * /Q0
S1  = /Q2 * /Q1 *  Q0
S2  = /Q2 * /Q1 * /Q0
S3  = /Q2 * /Q1 * /Q0
S4  =  Q2 * /Q1 * /Q0
S5  =  Q2 * /Q1 *  Q0
S6  =  Q2 *  Q1 * /Q0
S7  =  Q2 *  Q1 *  Q0
```

A.4.5 Gray code

With Gray code there is only one bit is different between adjacent codes and this is used to prevent spurious outputs occurring when changing between adjacent states.

```
S0  = /Q2 * /Q1 * /Q0
S1  = /Q2 * /Q1 *  Q0
S2  = /Q2 *  Q1 *  Q0
S3  = /Q2 *  Q1 * /Q0
S4  =  Q2 *  Q1 * /Q0
S5  =  Q2 *  Q1 *  Q0
S6  =  Q2 * /Q1 *  Q0
S7  =  Q2 * /Q1 * /Q0
```

A.4.6 One hot encoding

One hot encoding uses one output per state and is useful when the output is used to select other logic functions; it avoids the need for a decoding state as it uses one flip-flop for each state.

```
S0  = /Q3 * /Q2 * /Q1 * /Q0   ; power up state
S1  = /Q3 * /Q2 * /Q1 *  Q0
S2  = /Q3 * /Q2 *  Q1 * /Q0
S3  = /Q3 *  Q2 * /Q1 * /Q0
S4  =  Q3 * /Q2 * /Q1 * /Q0
```

A.4.7 State transitions

Following the state assignments section comes the state transition section, which identifies when a state changes and what state it will change to. Each state is identified by :=, followed by the condition which will trigger a change of state. The conditions are described in the conditions section. After the condition (in the next example: change_state) comes the change state symbol, ->, followed by the state to change to, as illustrated in the example. Multiple conditions can be included for each state by using the + symbol and they effectively operate as a CASE statement.

Example

```
S1  :=  change_state -> S2
S2  :=  change_state -> S0
        + go_up -> S3
        + -> S4
```

If no condition is given, then an unconditional default state is created, and in the above example, if conditions change_state and go_up are not true then the default will be to go to state S0. This is a local default state and overrides the global default state given if the DEFAULT_STATE option was included in the state header.

A.4.8 Input conditions

The input conditions follow the state transitions section and are identified by the reserved word CONDITIONS. Conditions which were previously used in the state transition section need to be defined here.

Example

```
conditions

    change_state = nothing*/clear
    go_up    = /nothing*/clear
```

A.4.9 Mealy state machine

Mealy state machines are similar to Moore state machines with one major difference, which is that the outputs at each state can be altered by the inputs to the state machine. This is achieved in the Mealy definition of a state machine by adding a fourth section, called the output transition definition section. This statement is not described in this appendix.

A.5 The simulation section

Once the logic model for the selected PLD has been produced, it is necessary to simulate it before programming the PLD. This is achieved by designing test vectors which will put the inputs through all the required states and match the simulated output against the predicted output. The simulator operates by applying the specified inputs, calculating the outputs and comparing with the user predicted outputs. These are sometimes known as test vectors. The simulation section is identified by the use of the reserved word SIMULATION after the logic has been defined. It consists of two types of command:

1) basic, 2) flow control.

A.5.1 Basic commands

There are two basic commands, one to control combinatorial inputs and one to control registers. These are:

1) SETF Sets the specified inputs, which can be vectors, as either logic high or logic low. If a design consists only of combinational logic this is the only command required.

2) CLOCKF Defines a LOW/HIGH/LOW pulse on the clock signal specified, which is connected to the clock input of the registers. Used in combination with SETF for registered logic designs.

A.5.2 Example of a combinational simulation section

```
simulation

setf value_1   value_2   value_3
setf value_1   value_2   /value_3
setf value_1   /value_2  value_3
```

To set a combinational input to logic high its signal name is used; to set it to logic low, a forward slash is used in front of its signal name to indicate it is inverted. The sequence in which the setf instructions are defined, specifies the order for the test vectors to be applied to the logic design. All the logic design inputs and outputs are recorded in a simulation file with the file extension .HST, which is called the simulation history file.

A.5.3 Example of a registered simulation section

```
simulation

setf      value_1   value_2   value_3
clockf    clk
setf      value_1   value_2   /value_3
clockf    clk
setf      value_1   /value_2  value_3
clockf    clk
```

In the above example of a registered design, a clockf instruction has been inserted after each set of input vectors. This enables the inputs to be specified and then clocked into the registers. Only the clock signal connected to the registers being used needs to be specified. If a device with multiple clocks is being used, both clock signals may need to be defined.

A.5.4 Additional basic commands

In order to help with the basic simulation sequence, there are five additional commands:

1) TRACE_ON To obtain only a sub-set of the complete simulation history file, the TRACE_ON command can be used to specify signals whose values are to be recorded in a trace file (with the file extension .TRF).

2) TRACE_OFF Used to close the trace file. Only one pair of TRACE_ON and TRACE_OFF commands is allowed within a simulation definition.

3) VECTOR Enables a sequence of signals to be treated as a vector. This is a definition only, so it must appear before the vector sequence is to be used.

4) CHECK Enables the specified signals to be checked for specific logic values. If there is a difference, an error is flagged in the simulation history file. If check is not used, then the user has to scrutinise the simulation history manually looking for the correct values.

5) PRLDF Preloads registers (and hence state machines) to known initial values. This is only a simulation command and is not recommended for maximum reliability designs. It is better to assume an initial unknown power-on state and then devise a simulation sequence which will move the design into the required state. Flow control can be used to achieve this.

A.5.5 Example of the additional basic commands

```
simulation

vector      data_bus[7:0] := [Q3,Q2,Q1,Q0]
trace_on    output_7
setf        value_1 value_2 value_3
clockf      clk
check       output_1 output_2 output_3
setf        value_1 value_2 /value_3
check       /output_1 output_2 /output_3
clockf      clk
setf        value_1 /value_2 value_3
clockf      clk
trace_off
```

A.5.6 Flow control

The basic commands given above enable any sequence of test vectors to be devised and applied. However, if a large logic design has been implemented, this can result in a very large set of test vectors which can be tedious to enter into the file. An alternative approach is to use flow control.

A.5.7 Flow control commands

Flow control automates the application of test vector sequences to the simulation, using structured high level language constructs, as listed below:

1) BEGIN END used to group simulation commands.
2) FOR TO DO repeats a sequence of test vectors for a specific number of times.
3) WHILE DO repeats a sequence of test vectors while a condition is true.
4) IF THEN ELSE applies flow control simulation sequence if the condition is true, otherwise it applies the alternative sequence.

A.5.8 Example of flow control commands

Any of the basic and additional basic commands can also be used within the flow control commands, provided that the correct syntax rules are obeyed.

```
simulation

vector    data_bus[7..0] := [Q3,Q2,Q1,Q0]
trace_on  output_7
setf      value_1   value_2   value_3
clockf    clk
check     output_1  output_2  output_3
setf      value_1   value_2   /value_3
check     /output_1 output_2  /output_3
clockf    clk
setf      value_1   /value_2  value_3
clockf    clk


;sets data_bus equal to all logic zeros for 5 clock pulses and then all
;logic ones for 5 clocks.
for index := 1 to 10 do
  begin
    if (index < = 5) then
      begin
        setf data_bus[7..0] := #b00000000
      end
    else
      begin
        setf data_bus[7..0] := #b11111111
      end
    clockf clk
  end

;An example of using the WHILE DO command.
while (/output_7 + /output_6) do
  begin
    setf output_4 output_5 output_6
    clockf clk
  end

trace_off
```

The use of basic, additional and flow control simulation commands enables complex and sophisticated sets of test vectors to be created easily and compactly. However, it is still the responsibility of the user to ensure that the test vector sequences are correct and that they are interpreted correctly, before considering the simulation of a logic design to be correct.

A.6 A language reference summary

A list of the reserved words used by MACHXL and PALASM is given below. Not all versions of MACHXL and PALASM will implement all of them.

A.6.1 Reserved words

3VOLT
5VOLT
BEGIN
CHECK
CHIP
CLOCKF
CMBFBK
CMOS_LEVEL
COMB
COMBINATIONAL
CONDITIONS
DEFAULT_BRANCH
DEFAULT_OUTPUT
DEFAULT_VALUE
DEFMOD
DELAYCLK
DO
ELSE
END
ENDMOD
EQUATIONS
FILE
FOR
FUSE
GND
GROUP
HIGH
HOLD_STATE
IF
INPUT
INST
INSTANCE
LAT
LATCHED
LATFBK
LOW
MEALY_MACHINE
MODULE
MOOR_MACHINE
NC
NEXT_STATE
NODE
OPEN_DRAIN
OPTIONS
OR
OUTPUT
OUTPUT_HOLD
PIN
PINFBK
PRELOAD
PRLDF
REG
REGFBK
REGISTERED
SECURITY
SETF
SIGNATURE
SIMULATION
STATE
STRING
THEN
TO
TRACE_OFF
TRACE_ON
TREGFBK
T_TAB
TTL_LEVEL
VCC
VECTOR
WHILE

Appendix B VHDL

B.1 Introduction to VHDL

VHDL is a hardware description language originally developed to improve the design of large integrated circuits by enabling an abstract description of the circuits' operation to be produced which could be simulated to identify any defects or omissions in the specification. Following this, the same description could be converted into a gate level description which could be implemented as an integrated circuit. Because of this the full name of the language is Very high speed integrated circuit Hardware Description Language. Though originally only designed for digital systems, the language has proved to be so flexible, useful and widely adopted, that extensions to incorporate analogue circuits have been devised and are already available in a limited form from some software developers.

B.1.1 Layout level

The language is capable of describing the same digital circuit at a number of different levels of abstraction. The lowest level, with the least abstraction, is the layout level which enables the description of the silicon and how transistors are formed and connected to be produced. This is where the language originated, but is now hardly used for this purpose, as there are now compilers which can convert higher levels of abstraction into much more efficient layout descriptions than most human designers can achieve. The layout level enables the effect of the layout on silicon to be investigated and incorporates timing and analogue effects, for it has to be remembered that although these systems are called digital, they are implemented using transistors which are inherently analogue devices.

B.1.2 Logical level

The most used level of description at present is the logic level, as this corresponds most closely to the construction of digital systems from standard building blocks of logic such as gates and registers. This is called the logic level and an appropriate compiler is used to convert the description into the layout level. The conversion process is known as synthesis and it has to be realised that not all valid VHDL descriptions at the higher levels of abstraction can be synthesised into working layout descriptions. It is usual to implement a set of guidelines which are designed to prevent the designer from creating unsynthesisable descriptions.

B.1.3 Register transfer level

As designers become more familiar with the concepts of designing digital systems with logic levels of abstraction, they are also wanting to use even higher levels of abstraction, as this enables a higher level of productivity to be achieved and a higher level of error-free designing. This is implemented in the register transfer level of abstraction, which does not consider what logic may be required to implement the design, but considers instead the functions to be implemented and what registers may be required. Effectively, it is a description of how data values are transferred from one store or register to another and how they are altered or amended during this process. Register transfer languages were originally developed during the design of microprocessors to help with the design of instruction sets. Each instruction of a complex microprocessor consists of several

smaller and simpler instructions which identify what data is transferred from what register to what other register and how it is altered. These instructions are sometimes known as microcode. Reduced Instruction Set Computers attempt to remove microcode and its associated time penalty, by making each micro-processor instruction out of a single register transfer level instruction.

Generally, register level designs are synchronous; that is, all actions take place at the same instant determined by some global signal known as the clock. Multi-clock systems are possible, as are asynchronous systems which do not use a global clock, and VHDL can be used to describe them all; however, it is much more difficult to design an error-free asynchronous digital system than a synchronous system. Therefore, most large digital systems are synchronous designs using only one global clock signal.

B.1.4 Behavioural level

The final level of abstraction available at present is the behavioural level, which does not even consider how any particular design could be implemented and does not make reference to logic gates or registers, or to the hardware architecture. It is purely concerned with the function to be implemented and is most useful for checking that the specification of a design is correct and complete. It is quite common for behavioural designs to be unrealisable, that is, they cannot be physically implemented, as the compilers to convert from behavioural to layout level do not yet exist.

This level is only really aware of the inputs, the expected outputs and the timing relationship between various input and output signals. The simulator is then used to verify that outputs change when and as required, determined by the input signals, the clock(s) and the past input states. This is a true abstract description of the hardware and its function. This level of abstraction is the most productive and is the one preferred by most designers; however, the difficulties of obtaining a layout description mean that it is not in widespread common use. The usual method is to implement a behavioural description to confirm the specification and then rewrite where necessary at the register transfer level (and if really necessary to the logic and layout levels) so that the design can be synthesised. At present, this level is usually only used to confirm the correctness and completeness of specifications of large systems, due to the synthesis problem. However, it is likely that before too long, true behavioural to layout level compilers will be devised and this level will then be increasingly used.

B.2 Data types and signals

To be useful, all abstract languages have to have the ability to manipulate data values which in turn requires the ability to define data variables. VHDL has a number of predefined data types, plus the ability for the user to define others. The predefined data types can be separated into four categories:

Boolean
numbers
characters
timing

Some of these data variables will correspond to the inputs and outputs of the design and can be considered to be electrical connections. However, some of them are intermediate variables, used to hold temporary values within parts of the design and which do not have an external connection; they are called signals. Signals can have the same data types as the input and output connections. To define an intermediate data variable, the reserved word signal is used, followed by the signal name and then the data type. A colon is used to separate the signal name from the signal data type and a semicolon is used to terminate the definition.

```
signal   weeks : integer;
```

B.2.1 Standard types

The simplest data type is the Boolean which has two values: TRUE and FALSE, and is typically used in conditional statements such as:

```
if (test = TRUE) then
  outp < = '1';
else
  outp < = '0';
endif
```

The definition of the intermediate variable or signal is as follows:

```
signal   test : boolean;
```

The most useful data types are the numbers and three number bases are pre-defined:

binary
integer
real

The binary values allowed: logic zero and logic one, are denoted by '0' and '1', which have single quotes either side and are similar to Boolean values. Integers are simply a sequence of numbers without any quote marks and real numbers are similar to integers except that they contain a decimal point. Both integer and real numbers can be positive and negative, although it has to be remembered that the numbers are manipulated in an underlying binary format based on the IEEE floating point and integer number representation standard. The definition of the signals is achieved as illustrated below:

```
signal   motor : bit;
signal   weeks : integer;
signal   average : real;
```

Variables can be allocated values as:

```
motor <= '0';
weeks <= 52;
average <= 11.23;
```

Alphanumeric characters can be defined in a similar manner, using either single character signals or multi-character sequences which are known as strings. Single character variables are defined using the data type character and sequences of characters using the data type string.

```
signal  initial : character;
signal  surname : string;
```

When allocating a single alphanumerical character, single quotes are used either side of the letter, whereas for strings, double quotes are used, as illustrated below:

```
initial <= 'R';
surname <= "Seals";
```

The final data type is different from all the others in that it is used to represent time rather than a value and can only really be used when timing constraints are being applied to a design. However, it is defined in exactly the same way as the other data variables, using the time reserved word.

```
signal  delay : time;
```

As the signal is designed to store time values which have a dimension, that is seconds, the values assigned to these signals must also include the time units as listed below:

```
fs    (femtosecond = 10-15)
ps    (picosecond  = 10-12)
ns    (nanosecond  = 10-9)
us    (microsecond = 10-6)
ms    (millisecond = 10-3)
sec   (seconds)
min   (minute = 60 seconds)
hr    (hour = 3600 seconds)
```

An example of using a signal of data type time is illustrated below:

```
delay <= 200 ns;
```

Note that some compilers may require a space to be present between the last digit of the numerical part and the first character representing the time units. Otherwise, it may be treated as a user defined variable or signal. If multiple signals are required which have the same data type, it is possible to define them on separate lines or on the same line using commas to separate the user defined names, as illustrated below.

```
signal  first, second, third : integer;
```

The above is exactly the same as the following:

```
signal  first : integer;
signal  second : integer;
signal  third : integer;
```

but the first method is quicker to input into the source file. However, if a comment is required to denote the purpose of each of the signals, the second method may be preferred.

```
signal  first : integer;      -- Used to denote the first person
                              -- in the queue.
signal  second : integer;     -- Used to maintain the second
                              -- person's age.
signal  third : integer;      -- Used to identify the third
                              -- character in the sequence.
```

Note that the double minus character sequence is used to indicate the beginning of a comment and the return at the end of the line is used to mark the end of the comment. Therefore, if comments are to extend over several lines of text, each new line must be marked with the double minus character sequence.

B.2.2 Bit vectors

VHDL includes a further predefined data type, called the vector, which uses the reserved word bit_vector and is used to describe a sequence of bit signal types. These are effectively one-dimensional arrays which are otherwise known as vectors. The use of bit_vectors enables groups of signals, usually buses to be manipulated as a whole rather than as individual bits, but still enables the individual bits to be accessed and treated as bit data types when required. As bit_vectors are sequences of bits, it is necessary to define the number of bits in the signal declaration. This is achieved by using the to and downto reserved words as illustrated below:

```
signal  data_bus : bit_vector(0 to 7);
```

The to and downto reserved words are used to identify whether the leftmost bit in the sequence is the most significant bit, using downto, or the least significant bit using to.

```
signal  data_bus : bit_vector(7 downto 0); -- This is an 8 bit data
                                           -- bus with the left
                                           -- most bit in the
                                           -- sequence being the
                                           -- most significant.

signal  other_bus : bit_vector(0 to 7);    -- This is an 8 bit data
                                           -- bus with the left
                                           -- most bit in the
                                           -- sequence being the
                                           -- least significant.
```

Once the order of the sequencing of a bit vector has been defined in this way, the same sequencing order must be used whenever the variable is used throughout the program. When using bit vectors, the size of the vector used is that given in the signal definition, unless an override size is given.

```
signal  a,b : bit_vector(3 downto 0)        -- Two 4 bit vectors
a < = b;                                    -- default size is used.
```

When using the override length within the assignments, both source and destination must be the same number of bits long.

```
signal  a : bit_vector(3 downto 0);
signal  b : bit_vector(7 downto 0);

a(3 downto 1) < = b(6 downto 4);
```

Constant values can also be assigned to vectors in a similar manner, provided that the number of bits in the constant is the same as the number of bits in the vector. Note that bit sequences are required by this activity, rather than individual bits, denoted by having double quotes at the beginning and end of their value, as follows:

```
signal  a : bit_vector(7 downto 0);

a(3 downto 1) < = "110";
```

The only exception to this is that the constant can have fewer bits than the vector, when the compiler will assume that leading zeros should be inserted, as shown below:

```
signal  a : bit_vector(7 downto 0);

a < = "110101";   -- The constant is only 6 bits long but the vector is
                  -- 8 so the compiler would insert two leading zeros.
```

Although these variables are being used as a single entity, it has to be remembered that they are in fact groups of bits and that using vectors is merely a convenient method of achieving this in a concise manner. The allocation of bits is determined by the sequence.

```
a (3 downto 0) < = "1101";
```

is equivalent to

```
a(3) < = '1';
a(2) < = '1';
a(1) < = '0';
a(0) < = '1';
```

and

```
a(3 downto 0) < = b(6 downto 3);
```

is equivalent to

```
a(3) < = b(6);
a(2) < = b(5);
a(1) < = b(4);
a(0) < = b(3);
```

One additional feature of bit vectors of interest is that signals originally defined as bit can be combined together to be assigned to a vector. This can be achieved in two ways, either using the concatenation operator, &, or grouping the bit variables together, using the open and close bracket symbols. Both these methods are illustrated below:

```
signal  a : bit_vector(3 downto 0);
signal  first, second, third, fourth : bit;

a(3 downto 0) < = fourth & third & second & first;
```

and

```
a(3 downto 0) < = (fourth, third, second, first);
```

which are both equivalent to

```
a(3) < = fourth;
a(2) < = third;
a(1) < = second;
a(0) < = first;
```

It should be noted that there are a number of other variations on how vectors can be defined and used which will not be explained in this simple description of the language. For a full description of the language the reader should consult one of the references given at the end of this appendix.

B.2.3 User defined data types

Although the predefined data types enable all the desired operations to be implemented, it is sometimes useful to be able to define new data types to make the program more concise, more understandable or more reliable. The user defined data types can help to produce programs that are more reliable by only allowing signals to hold specific values. The same effect can be achieved with the predefined data types, but they also allow unwanted values to be validly stored which is a potential source of error and hence the program is more unreliable. The reserved word type is used to identify that a user defined data type is to follow as illustrated below:

```
type state_type is (ready, blocked, running);
```

Type is followed by the user defined name for the new data type, which in the above example has been chosen to be state_type, which is then followed by the reserved word is and then the new data type names delimited by the brackets. Inside the brackets the names are separated by commas. State_type can now be used just like any of the predefined data types in signal definitions. Any signals defined in this way can only have the names specified in the type definition allocated to them, as illustrated below:

```
type state_type is (ready, blocked, running);

signal  status,high_status : state_type;

status < = ready;
high_status < = blocked;
```

The above user defined data types are known as enumerated types, as the compiler allocates a specific number to each of the names defined by the user within the brackets and then uses that number within the program. However, the user is unaware of this conversion process unless looking at specific outputs and logic values. The user defined numeric type enables the user to place limits on the predefined data types to achieve the improved reliability that is possible with reduced sets of signal values. These are defined in the same manner, except that the bracket containing the user specified names is replaced by the reserved word range followed by the allowed numeric values. An example using integers is given below:

```
type reduced_int is range 0 to 255; -- limits integers to 8 bit values.

signal  port_id : reduced_int;

port_id < = 255;
```

This particular user defined numeric type is not always supported by synthesis tools and is not often used in logic design, but it can be useful in behavioural designs when specifications are being validated. A number of other data types exist and methods of defining them which are beyond the scope of this simple introduction. Further information can be obtained from the references at the end of this appendix.

B.2.4 Standard logic IEEE 1164

Before a VHDL program can be compiled, it is necessary to specify any libraries that are required by the program. The library can contain any previously designed entity or data types and can be defined by the user. All programs require the inclusion of the standard library known as IEEE, as it was defined by the IEEE committee for VHDL which includes all of the predefined data types and the predefined logic operations. The library reserved word is used to indicate which libraries are being used. Multiple libraries can be defined on the same line provided that they are separated by commas. However, for the simple programs being designed here only the IEEE library will be included as illustrated below.

```
library IEEE;
```

Within the library there are a number of variations and the user has to indicate which ones are being used through the use reserved word as illustrated below:

```
library IEEE;
use IEEE.Std_logic_1164.all;
```

These two lines should be included at the beginning of all of the programs designed using this simple introduction and should enable all the programs developed to compile and simulate using any VHDL package. Further explanation of the library and use reserved words are beyond the scope of this introduction. See references for further information.

B.3 Operators

In order to be able to perform useful activities a wide variety of logical and arithmetic operators are available within VHDL, although only some of the more modern compilers may implement the less common ones. The most basic operator is assignment, which allocates the value on the right hand side of the assignment operator, which is the left angle bracket immediately followed by an equals symbol, to the signal or variable on the left hand side. Thus:

```
port_id < = 255;
```

allocates the constant value 255 (which would be converted to the binary "11111111") to the signal called port_id. The right hand side can be constant, another signal or variable, or a logical or arithmetic equation, provided that the compiler is able to determine that a constant value will result when the equation is implemented. Examples of this are:

```
first < = second;
second < = true;
```

The restrictions are that all the signals, variables or constants involved in an equation must all have the same data type, so the following would generate an error:

```
signal  port_id : integer;

port_id < = 25.5;
```

B.3.1 Logical operators

The standard logical operators are implemented, although they are represented with a word rather than a mathematical symbol, as the symbols are reserved for use as arithmetic operators. There are six basic operators as listed below which operate on bits, bit vectors and Boolean:

```
and
or
```

```
nand   -- not and
nor    -- not or
xor    -- exclusive or
not
```

VHDL compilers created after 1993 should also include the followed more specialised logical operators

```
sll    -- shift left logical
srl    -- shift right logical
sla    -- shift left arithmetic
sra    -- shift right arithmetic
rol    -- rotate left
ror    -- rotate right
```

The difference between shifting and rotating is that with shifting the bit moved out of the end of the signal is lost, whereas with rotating it is added to the other end of the signal. Logical shifts always add logic zeros into the vacant bit and arithmetic shifts consider the most significant bit of the value to be the sign and maintain that value. So a shift right arithmetic will keep the value of the most significant bit the same, but a shift right logical will insert logic zeros in the vacant most significant bit. It should be noted that shift left logical and shift right logical are equivalent and both add logic zeros into the vacant least significant bit. Examples of using the basic logical operators are:

```
first < = "1101";
second < = "0011";
third < = first and second;
fourth < = first and (not second or third);
```

B.3.2 Conditional operators

As well as the six basic logic operators, there are six conditional operators which enable comparisons between values to be made, the results of which can be used to control the operation of the logic. These types of operators are also known as relational operators. Generally, these operators will return Boolean results, that is, true or false, and are used by the conditional statements available within VHDL, such as IF THEN ELSE. These are high level statements which enable complex Boolean equations to be simply and easily implemented, thereby improving programmer productivity. The six operators are denoted by various combinations of symbols:

```
<      less than
<=     less than or equal
>      greater than
>=     greater than or equal
=      equal to
/=     not equal to
```

The use of <= for assignment differentiates between assignment and the test for equality which uses the equals symbol on its own, rather than allowing the compiler to

determine which should be implemented, as is common with other high level languages. An example using a conditional operator is:

```
signal  first, second : integer;
signal  biggest : boolean;

first < = 27;
second < = 34;
biggest < = (first < second);
```

Note that the use of the conditional operator returns a result which is Boolean from two signals which were defined as integer, allowing the result to be assigned to a Boolean signal.

B.3.3 Arithmetic operators

Where VHDL has a significant advantage over other hardware description languages is in the area of arithmetic operators. Most other HDLs do not incorporate arithmetic operators because of the difficulty the compiler has in converting the equations into an executable logic system. However, as VHDL is an international standard which has been designed to enable behavioural simulation for testing specifications where arithmetic operators are much more important, the compilers are able to handle arithmetic operators. However, it should be noted that not all VHDL compilers will implement all of the operators and some may limit their use to behavioural simulation rather than synthesising equivalent logic. The following arithmetic operators can usually be used in synthesisable designs:

+	addition
−	subtraction
*	multiplication
/	division

Note that all of these operators will work with integer or real numbers, but not a mixture in the same equation. The following additional operators can sometimes be used in behavioural designs but may not be synthesisable.

* *	exponential
abs	returns the absolute value of a number
mod	modulus
rem	remainder

All of these operators can be used in the same way as for any other mathematical equations and simplify considerably the conversion of numerical algorithms into executable digital logic systems. Examples of the use of the four basic operators are:

```
signal  numb_a, numb_b, numb_c : integer;
signal  first, second, third : real;
```

```
numb_a <= (numb_b + numb_c) / 2;
first <= 3.2 * second - third;
```

B.4 Concurrent constructs

One of the major differences between hardware description languages and computer languages is that the majority of computer languages are sequential, that is, the execution starts at the top of the program and executes each statement in turn. The order of the statements then becomes important. However, for HDLs it has to be remembered that the compiler will be producing a hardware implementation for each statement and these will all be executing in parallel, so the order that the statements are written in does not have an effect on the hardware synthesised or on the function of the hardware. For example, the following two program fragments are equivalent:

```
signal  numb_a, numb_b, numb_c : integer;
signal  first, second, third : real;

numb_a <= (numb_b * numb_c) / 2;
first <= 3.2 * second - third;
```

and

```
signal  numb_a, numb_b, numb_c : integer;
signal  first, second, third : real;

first <= 3.2 * second - third;
numb_a <= (numb_b * numb_c) / 2;
```

Further difficulties arise because the way the statements are written does affect the hardware synthesised even though the function of the logic may be equivalent. For example, the following statement:

```
first <= (second and third) or (second and fourth);
```

would be implemented using two AND gates and an OR gate. But it is functionally equivalent to the following:

```
first <= second and (third or fourth);
```

which only requires one AND gate and one OR gate. This saves one AND gate, enabling more logic to be implemented in the same device or a smaller device to be used. Although these two statements are functionally equivalent, there is a timing difference. For the first solution, all the signals have to pass through two logic gates and therefore incur two propagation delays. For the second statement, the second signal only passes through one logic gate and therefore its timing characteristic will be slightly different. The difference may or may not have an affect on some other part of the system which could be identified from the simulation. It is recommended that designers should *not*

incorporate implicit delays such as these, as they are too dependent on the compiler and the technology of the device used for implementation. Instead, they should explicitly include timing information using the wait statement and attempt to overcome possible problems due to propagation delays through the use of synchronous systems which coordinate all signal changes to a global clock.

B.5 Constructing a complete VHDL program

So far, only program fragments of VHDL have been considered in order to introduce the concepts and statements in a progressive sequence. To create a program which can be compiled and simulated, additional statements are required. The first of these is the entity statement which identifies the physical input and output signals that the logic function will have. These relate closely to the physical voltages that are connected to the device used. An example of an entity declaration to implement a 2-bit full adder is shown below:

```
entity  full_add_2bit is
port    (in1, in2, carry_in : in bit;
   sum, carry_out : out bit);
end full_add_2bit;
```

The entity reserved word is followed by the user defined name, followed by the reserved word is. On the next line the reserved word port is used to declare first the input signals and their data type, followed by the output signals and their data type. The reserved words in and out are used just before the signal data type identifiers. Finally, the entity declaration is completed by the reserved word end followed by the same user defined name used in the first line. All this is part of the approach VHDL takes of forcing the programmer to use structured programming methods as these are more likely to lead to a better design. The example given above has three input signals and two output signals and is represented in block diagram format in figure B.1.

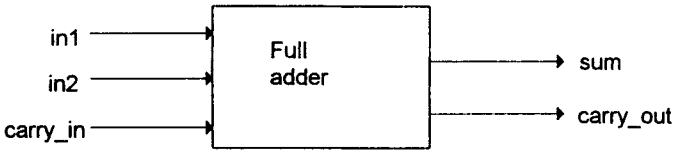


Figure B.1 Block diagram

The entity declaration is a block box definition that does not give any information about the internal operation. The internal operation is known as the architecture and should follow the entity declaration; it uses the reserved word architecture as illustrated below:

```
entity  full_add_2bit is
port    (in1, in2, carry_in : in bit;
   sum, carry_out : out bit);
end full_add_2bit;
```

```
architecture struct of full_add_2bit is

begin
  sum < = (not in1 and not in2 and carry_in) or
        (not in1 and in2 and not carry_in) or
      (in1 and not in2 and not carry_in) or
      (in1 and in2 and carry_in);
  carry_out < = (not in1 and in2 and carry_in) or
              (in1 and not in2 and carry_in) or
              (in1 and in2 and not carry_in) or
              (in1 and in2 and carry_in);
end struct;
```

The reserved word architecture is followed by a user defined name followed by the reserved word of, then the user name for the entity, followed by the reserved word is. Following this the rest of the architecture is defined between the begin and end reserved words. The end reserved word is followed by the user defined name for the architecture to complete the definition. This particular implementation of the full adder uses the full Boolean equations; alternative architectures which would have the same entity declaration could use the half adder approach to achieve the same functionality. If additional signals are required, they are defined between the line containing the architecture and the begin reserved words. For example, if the user decided to remove some of the redundancy by creating a single instance of the (in1 and in2 and carry_in) logical operation by using an intermediate signal, the following program would be derived:

```
entity   full_add_2bit is
port     (in1, in2, carry_in : in bit;
    sum, carry_out : out bit);
end full_add_2bit;

architecture struct of full_add_2bit is

signal   all_and : bit;

begin
  all_and < =(in1 and in2 and carry_in);
  sum < = (not in1 and not in2 and carry_in) or
        (not in1 and in2 and not carry_in) or
      (in1 and not in2 and not carry_in) or
         all_and;

  carry_out < = (not in1 and in2 and carry_in) or
              (in1 and not in2 and carry_in) or
              (in1 and in2 and not carry_in) or
               all_and;
end struct;
```

Again, both of these programs are functionally equivalent, but there may be small timing differences; there are also the differences in the number of logic gates used and how they are connected.

B.6 High level language constructs

There are three high level language constructs available within VHDL, designed to make the creation of programs easier. However, it should be remembered that these statements can be difficult for the compiler to convert into digital logic and some variations are best not used. For behavioural designs which are not going to be synthesised into logic, there are no restrictions on their use. The three constructs are:

IF THEN ELSE
CASE
FOR TO LOOP

and these are explained in the following sections.

B.6.1 The IF THEN ELSE construct

The syntax of the IF THEN ELSE construct is as given below with the ELSE being optional.

```
if (boolean test is true) then
  statements;
end if;
```

The first example does not have the optional ELSE part of the statement, whereas the following example includes it. Note that the end of the if statement is identified by the reserved word if after the last end.

```
if (boolean test is true) then
  statements;
else
 other_statements;
end if;
```

The if construct is useful for creating multiplexers and an example is:

```
if (select = '1') then
  out1 <= in1;
else
 out1 <= in2;
end if;
```

A demultiplexer can also be created in a similar way, although it must be remembered that all the outputs should be explicitly defined at all times if all output signal values are to be known at all times. Otherwise, the optimiser in the compiler may treat it as a don't care and some unpredictable results can occur.

```
if (select = '1') then
  out1 <= in1;
  out2 <= '0';   -- unused outs set to logic zero.
else
 out1 <= '0';
 out2 <= in1;
end if;
```

There is also the possibility of creating nested if constructs using the reserved word case as illustrated below. The following example also demonstrates the use of the conditional operators on integer numbers when determining the result of the Boolean test.

```
if (numba > 0 and numb <= 5) then
  out1 <= '0';
  out2 <= '0';
else if (numba > 5 and numb <= 8) then
  out1 <= '1';
  out2 <= '0';
else
  out1 <= '1';
  out2 <= '1';
end if;
```

However, a more efficient method of implementing the same functionality as using nested if constructs is to use the CASE construct.

B.6.2 The CASE construct

The case construct allows one of a different number of statements to be executed, determined by the value of an expression. The expression can be any signal capable of having multiple values, such as a bit_vector or an integer. However, real numbers should not be used due to the problems of obtaining a correct match. Even bit data types can be used, but as they have only two possible values, this would be equivalent to an if construct and therefore would have no advantage in being used. The syntax of the case construct is given below and starts with the case reserved word, followed by the expression (usually treated as an integer but can be a bit_vector) followed by the reserved word is.

```
case expression is
  when value1 =>
    statements;
  when value2 =>
    other_statements;
  when choice3 =>
    more_statements;
end case;
```

On the next line the reserved word when is used to identify what value the expression is being tested for. After the value there are the reserved symbols equals immediately

followed by the right angle bracket (= >) followed by the other VHDL construct or constructs required to be executed for that particular situation. The when reserved word is used to identify further values that the expression can have and what constructs should be executed. After the last when there is the reserved word end followed by the reserved word case. All possible values of the expression must be included otherwise the operation of the logic will be unpredictable. Therefore either limited integer signals must be used or the others reserved word following the last when as illustrated below:

```
case expression is
  when value1 = >
    statements;
  when value2 = >
    other_statements;
  when choice3 = >
    more_statements;
  when others = >
    final_statements;   -- Any values of the expression not covered
end case;               -- previously are detected here.
```

An example using the case construct for a 1-to-4 demultiplexer is:

```
case channel is
  when 0 = >
    out1 < = in1;
    out2 < = '0';
    out2 < = '0';
    out3 < = '0';
  when 1 = >
    out1 < = '0';
    out2 < = in1;
    out2 < = '0';
    out3 < = '0';
  when 2 = >
    out1 < = '0';
    out2 < = '0';
    out2 < = in1;
    out3 < = '0';
  when others = >
    out1 < = '0';
    out2 < = '0';
    out2 < = '0';
    out3 < = in1;
end case;
```

The use of the others reserved word with the last choice is an example of good design technique. The expression should never have any values other than 0, 1, 2 and 3 but, by adding the others to the last when, if any alternative value such as 5 accidentally appear, the logic will still operate. The decision as to what should happen with the spurious

values is decided by the user. In this example, it has been decided that it will always be out3. An alternative, which might be preferred in some situations, would be to turn all of the outputs off, as illustrated below and activate an error signal. Allocating error signals in this way consumes hardware resources, but is useful for systems where errors have to detected and recorded so that user intervention can take place. This is often used in high reliability systems.

```
case channel is
  when 0 = >          -- selects first output channel
    out1 < = in1;
    out2 < = '0';
    out2 < = '0';
    out3 < = '0';
    error < = '0';
  when 1 = >          -- selects second output channel
    out1 < = '0';
    out2 < = in1;
    out2 < = '0';
    out3 < = '0';
    error < = '0';
  when 2 = >          -- selects third output channel
    out1 < = '0';
    out2 < = '0';
    out2 < = in1;
    out3 < = '0';
    error < = '0';
  when 3 = >          -- selects fourth output channel
    out1 < = '0';
    out2 < = '0';
    out2 < = '0';
    out3 < = in1;
    error < = '0';
  when others = >     -- error detection test
    out1 < = '0';
    out2 < = '0';
    out2 < = '0';
    out3 < = '0';
    error < = '1';    -- An error situation has occurred
end case;
```

B.6.3 The FOR TO LOOP construct

The for loop allows an efficient method of repeating a sequence of statements a known number of times. Examples might be to set all the bits in a bit_vector to logic 1 or to output an incrementing integer. The syntax of the construct is given below and starts with the for reserved word followed by a signal name, then the reserved word in followed by the range of values that the signal can have:

```
for signal_name in lower_limit to upper_limit loop
  statements;
end loop;
```

This is then followed by the statements which are to repeatedly executed and then the reserved words end and loop.

B.7 Processes and sequential statements

So far, only statements which are all executed in parallel have been considered, but VHDL has the facility to execute statements sequentially, as most other programming languages do, using the process construct. The process construct sets aside a group of statements to be execute sequentially, starting with the one nearest the top and ending with the one nearest the bottom of the declaration. If more than one process is declared in a program, the processes operate in parallel but within the processes the statements are executing sequentially. The process construct is a complex one and only a simple example of the syntax and examples will be given. Further details can be obtained from the references. The process is declared using the reserved word process followed by the begin reserved word and then the statements to be executed. The declaration is completed by the reserved words end and process. An example of the syntax is:

```
process

begin
  statements;
end process;
```

One feature of processes that is particularly useful is that the declaration may contain a sensitivity list which specifies that if any of the listed signals alters the process will be executed. This enables a number of processes to be declared, but they will only execute if one of the signals is changed. An example of a process with a sensitivity list is:

```
process (first, second, third)

begin
  first <= second and third;
end process;
```

Whenever any of the signals, first, second or third is altered, the process will execute.

B.8 Clocked designs

So far all of the designs have been combinational, that is, they have not relied on the presence of a global synchronising signal. VHDL does not explicitly require synchronous designs to be specified, but instead requires the designer to indicate which signal will be used to coordinate actions. One method of achieving this is to use the wait command.

This is indicated by the use of the wait reserved word then the until reserved word, followed by the condition to be waited for. The syntax is:

```
wait until (my_sig = '1');
```

This construct will cause the function to remain at that point in the program until the signal my_sig becomes equal to a logic 1. When compiled and synthesised this is implemented as a registered signal with my_sig being the clock connected to the register. An example of a two bit counter is:

```
entity twobit_counter is
  port( clk : in bit;
    cnter : out bit_vector(1 downto 0));
end twobit_counter;

architecture struct of twobit_counter is

signal state1 : bit_vector(1 downto 0);

process

begin
  wait until (clk = '1');
  case state1 is
    when "00" =>
      cnter <= "01";
      state1 <= "01";
    when "01" =>
      cnter <= "10";
      state1 <= "10";
    when "10" =>
      cnter <= "11";
      state1 <= "11";
    when others =>
      cnter <= "00";
      state1 <= "00";
  end case;
end process;
end struct;
```

B.9 Other constructs

So far, only a limited number of VHDL constructs have been described in a simplified way. There is a significant number of other constructs which have not been described which are beyond the scope of this introduction. Many of them are related to creating behavioural designs which it may not be possible to synthesise into hardware. Because

of this they are not described. However, further information can be obtained from the references given below or from any good book on VHDL design.

B.10 A complete VHDL program

A simple complete VHDL program which implements a simple memory address decoder for a microprocessor based computer is listed below.

```
-- R C Seals
-- 23 July 1996
-- University of Greenwich
-- A simple address decoder

library IEEE;
use IEEE.Std_logic_1164.all;

entity decoder is
  port(a15, a16, a17, a18, a19 : in  bit;
       cs_rom1, cs_rom2, cs_ram1, cs_ram2 : out bit);
end decoder;

architecture micro of decoder is

begin
  cs_rom1 < = not (a15 and a16 and a17 and a18 and a19);
  cs_rom2 < = not (not a15 and a16 and a17 and a18 and a19);
  cs_ram1 < = not (not a15 and not a16 and not a17 and not a18 and not a19);
  cs_ram2 < = not (a15 and not a16 and not a17 and not a18 and not a19);
end micro;
```

B.11 References

1. IEEE Standard VHDL Language Reference Manual, IEEE Std 1076-1987, New York, IEEE publication No. SH11957.

2. IEEE Standard VHDL Language Reference Manual, IEEE Std 1076-1993, New York, IEEE publication No. SH16840, ISBN 1-55937-376-8.

Glossary

Adder	Logic device which adds two binary numbers together to form a sum, with optional carry.
Algorithm	A list of statements which when executed sequentially will implement the required process.
ALU	Arithmetic Logic Unit. A collection of digital circuits which implement the basic logic and arithmetic operations such as AND, OR, NOT, add, subtract and so on.
AND gate	A collection of transistors which implements the Boolean AND function.
AND-OR architecture	A standard PLD circuit construction, with all the inputs being connected to AND gates and the outputs of the AND gates being connected to the inputs of OR gates. The outputs from the OR gates implement the AND-OR Boolean function.
Antifuse	When an antifuse is programmed, a connection that was previously 'not conducting' is made to conduct. (The opposite to a fuse.)
Architecture	The internal interconnection of a digital system indicating how data is transfered between various functions.
ASIC	Application Specific Integrated Circuit. A name used to describe an IC which has been designed at transistor level to implement a specific function, and cannot be used for anything else.
Asynchronous	Used to describe events which can occur at any time, that is, they are time independent.
Ball grid array	Method of making solder connections between an IC and a PCB, using small balls of solder preformed on the IC.
Barrel shifter	Efficient form of shift register.
Benchmark	A standardised method of 'doing something'. The aim is to provide a method which allows different processes and

	designs to be compared. To be widely accepted, they should be manufacturer independent.
Bi-directional	Describes either a connection or a logic circuit which allows electrical signals to travel in either direction.
Bipolar	A method of constructing transistors, which produces fast switching transistors but which consume a relatively high amount of power, most of which is dissipated as heat.
Boolean	Usually used to describe the state of some variable which can have only two values, called either true and false, or for digital circuits, logic 1 and logic 0.
Boolean logic	A method of combining Boolean values using a set of logical rules.
Buffer	A logic circuit which is connected between a signal source and its destination to provide protection and additional current and/or voltage capability.
Bus	A collection of digital signals with a consistent function, for example, memory address.
CAD	Computer Aided Design. In this context, used to describe the software tools used to create the logical designs and then convert them so that they can be programmed into the PLDs and FPGAs.
Clock	A repeated digital waveform with a regular mark and space, which has fast rise and fall times that is used to synchronise logic devices in different parts of a system.
CMOS	Complementary Metal Oxide Semiconductor. A semiconductor production process which creates digital circuits that have a very low power consumption.
Combinational	Used to describe a logic circuit in which the instantaneous output(s) is completely and always determined by the instantaneous inputs. Such systems do not have any registers and have no method of memory storage.
Combinatorial	See combinational.

CQFP — Ceramic Quad Flat Pack. A type of IC encapsulation made from a ceramic material which has external connections along each of four sides (usually a square package shape is used).

Critical path — The path within a digital circuit along which a signal will travel which has the most important parameter, for that particular design. Typically, time is the most important parameter, so the longest path is the critical one as high speed operation is required.

Custom — Design of a specific digital circuit for a specific function. See also ASIC.

D type — A type of flip-flop where the output is determined solely by the value present at the input at the instant of a valid clock pulse. The output logic value remains valid until the next clock pulse.

Datapath — The physical path that signals take in a circuit.

DRAM — Dynamic Random Access Memory. A form of read/write memory which needs constant refreshing to maintain the correct data values, but which requires only low power, and uses a minimal number of transistors to implement. It is particularly suited for the production of very large, cheap, solid state memories for computer systems.

EDIF — Electronic Data Interchange Format. A standardised representation of data which is independent of specific manufacturers and is designed to allow the transfer of information between otherwise incompatible software systems.

Electronically configurable — Where the logic function implemented by a programmable logic device can be changed electronically, without having to remove the device from the PCB so that is can be erased by UV light or other means.

Enable — A signal which allows or disallows a digital signal to output from some part of a digital system to another part of the system.

EPROM — Erasable Programmable Read Only Memory. A component used to store binary values in a large number of internal locations which is non-volatile. By exposing the memory cells

	to high power UV light, the contents of the memory locations can be erased so that new values can be stored there.
Exclusive OR gate	A logic circuit which implements the Boolean exclusive-OR logic function.
Fan-in	The maximum number of digital signals that can be connected together at one input of a digital circuit.
Fan-out	The maximum number of digital signals that can be connected together at one output of a digital circuit.
FET	Field Effect Transistor. A low power consumption, easily made transistor which does not occupy too much silicon, allowing large numbers of transistors to be placed on a single IC.
Flip-flop	A logic circuit which has two stable output logic states, logic 0 and logic 1. By applying the correction stimulus to the inputs, the output can be made to change from one stable state to another. When the input stimulus is removed, the output remains at the last programmed value. A flip-flop is the basis of memory components, with each flip-flop able to store one bit of information.
Footprint	The area of PCB occupied by a particular IC.
FPGA	Field Programmable Gate Array. Array of blocks of logic which can be programmed to implement a wide selection of logic functions, with programmable interconnections.
Fuse	An originally conducting connection point which can be made to become open circuit permanently by the application of large currents for short periods of time, until the connection literally melts and evaporates. The process is non-reversible.
Gate	The smallest digital circuit possible, usually an AND or OR gate.
Glue logic	The various gates used to make the necessary timing and logic conversions to enable two or more larger components to be effectively connected together.

Granularity	A description of the relative smallness of the basic logic building block of a programmable logic device.
Input	The physical point at which the digital voltages and signals enter the IC packaging.
Interconnection	Physical low impedance path between logic elements.
Invertor	Logical NOT operation.
JTAG	Joint Test Action Group. An IEEE standard describing a method of testing the interconnections and operations of a PCB based digital system.
Latch	A registered element where the output changes with the input while the enable is active high. When the enable is deactivated, the last valid logic level on the input is maintained on the output until the next activation.
LCA	Logic Cell Array (Xilinx name for FPGA).
Library	Collection of similar elements and basic building blocks for logic system designs.
Look-up table	An addressable memory system used to convert one value into another.
Macro	Group of statements or actions to be executed.
Mapper	Converts the logical operations required by the design into a suitable format for programming on to the physical component to be used.
Mapping	The process of converting the logical design and placing it into the chosen component.
Minimisation	The process where Boolean equations are reduced to their most compact and efficient form.
MOSFET	Metal Oxide Semiconductor Field Effect Transistor. A robust and low power implementation of a FET.
Multi-chip module	Two or more silicon chips bonded on to a substrate and connected using gold or silver loaded paints.

Multiplexer	Logic function to convert multiple input signals into a single output signal.
Multiplier	A logical function to perform the arithmetic multiply operation.
Netlist	A description of the interconnections between the components or logic blocks in an electronic system.
NRE	Non Recurring Engineering costs.
One hot coding	A method of designing state machines where there is a separate register for each specific state.
Optimisation	The process during which both the speed of operation of the logic system is increased and the number of logic gates needed to implement the logic system is decreased until a satisfactory compromise is reached. Typically, the fewer gates there are, the slower the system will operate.
OR gate	Logical OR operation.
Output	The result of a logical operation.
Packaging	The physical protection added to an integrated circuit to provide mechanical strength, protection from the environment and heat dissipation.
Partitioning	The process where a large design is separated into a number of smaller interconnected modules.
PGA	Pin Grid Array, a method of connecting an IC package to the PCB.
Pin out	The allocation of internal logical functions on a component to the physical pins on the packaging.
Place and route	The process where the required logical functions are allocated to the available logic operations on the physical component and their subsequent interconnection.
PLD	Programmable Logic Device. A limited array of blocks of logic which can be programmed to implement a small selection of logic functions.

PLCC	Plastic Leadless Chip Carrier; a form of IC packaging which does not have any pins but which is soldered directly to the PCB.
PQFP	Plastic Quad Flat Pack; a form of IC packaging which has a low profile, has connections along all four sides of the device and is constructed from plastic.
Primary memory	Memory used by a computer to store programs (and data) while being executed.
Product term sharing	The process whereby logical OR terms from adjacent logic functions within an FPGA are combined in order to produce a larger product term equation than either of them can produce individually.
PROM	Programmable Read Only Memory; a non-volatile form of memory component which can be programmed only once.
Propagation	The time taken for an electronic pulse or signal to travel from some identified start position to some other identified stop position.
Prototype	Initial construction of a design in order to determine if there are any unforeseen problems.
Pull-down	A component, usually a resistor, which is used to force an undriven logical signal to a logic low.
Pull-up	A component, usually a resistor, which is used to force an undriven logical signal to a logic high.
RAM	Random Access Memory; an inaccurate, but commonly used term to describe a volatile memory component capable of having its internal data accessed in any sequence and altered any number of times during the digital system operation.
Reconfigurable	A type of logic function which can be altered as the system is executing.
Register	The smallest type of memory storage, where the output is altered on the active edge of the clock signal and remains valid until the next active edge.

Reset	The process whereby logic signals are returned to their initial power on values.
ROM	Read Only Memory; a form of non-volatile memory component.
Routing	The selection of the interconnection between logic functions that have been mapped to a target device.
Secondary storage	Off-line program and data storage, usually hard disks.
Self-timing	A description of a type of asynchronous logic system design.
Simulation	The process of imitating the operation of a logic system before it is constructed in order to verify that it will operate as required.
Slew rate	The rate of change of an electronic signal, usually a voltage.
Slew rate limited	A description of a logic output which limits the rate at which the signal can change.
SRAM	Static Random Access Memory; a type of RAM which does not require a clock signal in order to maintain its data values.
SSI	Small Scale Integration; a term used to describe logic components containing less than 100 logic gate equivalents.
State machine	A term for a diagrammatic method of describing the operation of a clocked or synchronous digital system, where each stable set of outputs is a state. The transfer of the outputs from one stable state to another is determined by the description.
Sum of products	A term used to describe the Boolean equation output by an AND-OR logic gate structure.
Synchronous	Description of a system where all the outputs change at the same time, determined by a global clock signal.
Synthesis	The process of converting a program description of a digital system into the hardware to implement it.

T type	A toggle register, where the output changes to the opposite logic state at every clock pulse.
TTL	Transistor-Transistor Logic; a term used to describe the output configuration of transistors on a logic device implemented using bipolar transistors.
Tri-state	A logical output which is capable of being in one of three valid states, logic 0, logic 1, or high impedance.
UNIX	Operating system in widespread use, particularly for work stations.
Usable gates	A description of the number of equivalent logic gates in a digital component that can be utilised within a design.
Verification	The process of confirming that a product meets the original specification.
VLSI	Very Large Scale Integration; used to describe logic systems containing more than 10,000 logic gate equivalents.
Workstation	Powerful personal computer with high resolution display, large primary memory and very large secondary storage.
Worst case	The combination of parameters and situations which causes the worst possible result to occur.

Index

22V10 32, 63, 65, 214
3 V logic systems 45
3 volt operation 116
5 Volt programming 147
7 segment decoders 195

Abstraction 121
Adder 166, 191
Algorithm 110
Ambient temperature 20, 34
Analogue array 46
AND-OR array 65
Anti-fuse 49
Anti-fuses 104, 106
Antifuse links 128
Application specific integrated circuit 213
Application specific integrated circuits 45
Application Specific Integration Circuits 7
Architecture 63, 102, 106, 119, 142
Arithmetic function 118
ASIC 236, 241
ASIC design 31
Assemblers 64
Asynchronous 65, 240
Asynchronous designs 239
ATPG 132, 238, 241
Automated migration 238
Automatic test equipment 57, 128
Automatic test pattern generation 132
Avalanche injection 49
Average Benchmark capacity 145
Average Benchmark Speed 145

Back annotation 239
Back-annotated 135
Back-annotation 131
BCD 217
Bed of nails 54
Bed-of-nails testing 148
Behavioural desciptions 24
Behavioural retiming 124
Benchmarking 140
Bi-directional buffer 104
Binary counter 187
Bonding and mounting 10
Boolean 118, 135, 192
Boolean equations 24, 68, 69, 215
Boundary conditions 77, 173, 196
Boundary scan 16, 17, 54
Buried inputs 64
Buried macro cells 27
Buried inputs 2
Burn in 18

Capacitance 19
Capacitors 36
CASE 163, 191
Case studies 30
Ceramic material 13
Chip carrier 16
Clock 64
Clock driver networks 106
Clock paths 118
Clock skew 106, 118
CMOS 16
CMOS RAM 53
Combinational 64, 75
Combinational designs 30
Combinational logic 63, 103
Combinational outputs 2
Commissioning 118
Comparators 116
Compiler 215
Compilers 64
Complex PLD 3, 65
Complex PLDs 29, 33
Complex programmable logic devices 67
Compression algorithm 111
Conductors 36
Configurable Logic Block 5
Cooling 20
Cost 131, 143, 162
Counter 197, 200, 211
Counters 26

Critical path 143
Critical paths 76
Cross-overs 36
Custom designed circuits 46
Czochralski crystal pulling 9

D flip-flop 106
Data retention 52
Debugging 128, 153
Decompiling 75
Demultiplexer 190
Design 153
Design for testability 240
Designing Test Vectors 75
Deterministic timing 30, 111
Development software 68
Development systems 112, 132, 140, 150
Die size 213
Dielectric 37
Diffused capacitors 37
Diffused collector transistor 38
Diffusion 12, 36
Digital signal processing 118, 148
DIL 214
Diodes 38
Dual-In-Line 13, 63
Dynamic hazards 134
Dynamic reconfiguration 110

EDIF 120, 128, 130
EEPROM 1
Electrically alterable ROM 51
Electrically erasable PROM 51
Electromagnetic interference 149
Emitter coupled logic 41
Emulator 153
Encapsulation 52
Epitaxial deposition 10, 36
EPROM 1, 6, 32, 209
Erasing 127
Espresso algorithm 122
Evaporation 12, 37
Exporting 129

Fabrication process 239
Fan-out 108
Fault coverage 239
Feedback 63
Feedback path 64
Field effect transistors 39
Field programmable gate array 213
Field Programmable Gate Arrays 1, 140, 218
Fitter 25, 71, 120
Fitters 112
Flexibility 162
Flip-flops 26
Formal verification 124
FPGA 58, 70, 102, 110, 126, 131, 189
FPGA emulator 128
Full-custom 236
Functional 26
Functional designs 24
Functional simulation 29, 102
Functionality 213
Fuse 22, 48
Fuse array 65
Fuse checksum 74
Fuse map 68, 72
Fuses 1
Fusible links 128

Gate array 46, 236
Gate counts 102, 141
Gate densities 213, 215
Gate level designs 26
Gate level equivalence 29
Gate level simulation 102
Gated clock signals 239
Germanium 9
Glitches 240
Global clock 108
Gold plated pins 15
Good design practice 239
Gray code 122, 174, 211, 241
Gull wing 15, 19

Hardware description languages 68, 215
Hazards 240
HDL 22, 121, 162
Heat dissipation 16
Heat sink 16

Hermetic 21
Hidden nodes 241
Hierarchical 121
High level language 68
Humidity 17
Hybrid circuits 7
Hybrid Integrated circuit 9

I/O macro cells 27
I/O modules 105
IEEE 143, 238
IEEE standard 17, 26, 54
Importing 129
Impurity gradient 37
In-circuit programming 71
In-system programming 147
Inductance 19
Insertion 15
Insertion forces 15
Integer arithmetic 200
Integrated circuit 7
Integrated injection logic 41
Integrated tool suite 119
Intellectual property rights 125
Interactive formal verification 125
Interconnection 27
Interconnections 104
Interconnects 106
Ion implantation 9

J-K type 106
JEDEC 23, 72, 100, 128, 216
Joint Test Action Group 54
JTAG 54, 72, 115, 117, 126, 128, 145, 148

Large pins counts 16
Large scale integration 8
Lateral transistor 43
Libraries 26, 121, 123
Library of parameterised models 123
Logic design 31
Look-up table 114, 117

MACHXL 69, 112, 187, 220, 230
Macro architecture 237
Macro cell structure 27
Macrocell 65
Macrocells 63, 71, 111
Macros 26
Mask 236
Master slice 46
Maximum operating frequency 141
Mechanical shock 13, 18
Medium scale integration 7
Megafunctions 118, 123
Metal oxide capacitors 37
Metal oxide semiconductor 41
Metallisation layer 236
Microcontrollers 118
Microelectronics 1
Microprocessor 123
Microprocessors 118
Migration 213, 228, 230, 236, 239
Model library 238
Monolithic 7
Moore's Law 213
MOS transistors 6
Moulded plastic 21
Mounting 12
Multiplexer 66, 108, 189
Multiplexers 26
Multiplier 170

Nailhead bonding 13
Netlist 119, 122, 129, 131, 229, 238
Noise 117
Non Recurring Engineering 229, 236
Non-deterministic place and route 102

Ohmic contacts 12
One-time programmable 71
One-time programming 146
Operating environment 13
Operating frequency 132
Optimisation 154
Optimisers 215
Oxidisation 10, 36

Packaging 12, 149
Pad driver 105
PALASM 24, 69, 111, 219, 220
Parasitic capacitance 44
Parasitic capacitors 19
Partial reconfiguration 110
Partition 125
Passive components 36
PCB 162, 214
PCB area 143
Photo-etching 11, 36
Photoengraving 10
Photographic mask 11
Pin damage 15
Pin Grid Array 13
Pin probing 16
Pin spacing 128
Pin through hole 13
Place and routing software 69
Placement 27
Plating 37
PLCC 19
PLD 131, 162
PLPL 32
Portable 131
Post layout simulation 239
Post-routing simulation 135
Power 162
Power consumption 131, 143, 213
PREP 144, 154
Product term 66, 117
Product term sharing 71
Programmable logic 1
Programmable logic device 46
Programmable Logic Devices 1, 59, 63, 140, 213
Programmable read only memory 46
Propagation delay 27, 42, 122, 142
Propagation delays 5, 64, 68, 102
Prototyping 140

RAM 145
Reconfiguration 110, 137
Reprogrammable 71
Reduced Instruction Set Computer 162, 181, 209
Register counts 141
Register rich architectures 102
Registered 75
Registered PLDs 2
Registers 63, 64, 133
Reliability 17, 131, 149, 162, 169
Reprogrammability 146
Reset signal 174
Resistors 36
Ripple carry 166
RISC 181, 209, 211
Robustness 240
Routing 27
Routing capability 142
Routing delays 237, 240

Scan chain 239
Scan chains 133
Schematic 22, 118, 120, 150, 215
Schematic based migration 237
Scribing 10
Semiconductor 7
Sequencer 174
Sequential 2, 64
Sequential logic 103
Silicon 9
Simulation 70, 102, 120, 125, 128, 163, 189, 214, 241
Simulator 68
Simulators 24
Sine wave generator 205, 212
Skew 111, 148
Slew 117
Small scale integration 7
Software 118
Solder bump 16
Solder faults 16
Specification 162
Speed of operation 102, 213
Sputtering 37
SR flip-flop 106
SRAM 116
Standard cell 46
Standardisation 102
Standards 128
State diagram 64, 215
State machine 56, 101, 162, 188, 211, 241

State machine testing 89
State machines 69
Static timing 131, 135
Step Motor Controller 178, 187, 202
Structural 26
Stuck at 0 132
Stuck at 1 132, 135
Stuck at one 77
Sum-of-products 2
Surface mount 13
Surface mount packages 15
Symbols 121
Synchronous 65
Synchronous circuits 239
Synthesis 26, 29, 135
Synthesiser 119

T type flip-flops 106
Temperature coefficient 37
Test pads 55
Test strategy 162
Test vector 73, 101, 120
Test vectors 30, 70, 132, 162, 169, 177, 184, 187, 189, 195, 202, 211, 214, 216, 228, 241
Testability 132
Thermal creep 15
Thermal cycling 19
Thermal resistance 20, 34
Thermal shock 18
Thermally generated leakage 50
Thermo-compression 13
Thick film 7
Thin film 7, 8
Thin film resistors 37
Time to market 214
Timing violations 135
Transistor-transistor logic 41
Transistors 38
Transmission checksum 74
Tri-state bus 117
Tri-state output 104
Triple diffused transistors 39
Truth table 164
Truth tables 69
Tunnelling 51

Ultra-violet light 49
Unix 119
Utilisation 71, 122

Validation 238
Vertical transistor 43
Very Large Scale Integration 8
VHDL 22, 68, 102, 122, 128, 151, 160, 189, 211, 214, 219, 230, 240
Video digitiser 216

Walking logic 0 77
WARP2 122
Waveform test vectors 30
Windows 119
Wired logic 43